Archivo: { Colección de datos almacenados juntos bajo un nombre común }.

- Almacenan información de manera permanente en dispositivos de almacenamiento de memoria secundaria.
    - La información puede ser tanto programas como datos que serán utilizados por programas.
- Para trabajar con archivos se utilizan, aparte de otras operaciones, unas operaciones básicas que son: *abrir, cerrar, escribir y leer*.
- Los archivos pueden ser de *lectura* (archivo de entrada) o de *escritura* (archivo de salida). El tipo de acceso determinará las *operaciones disponibles* para el archivo.

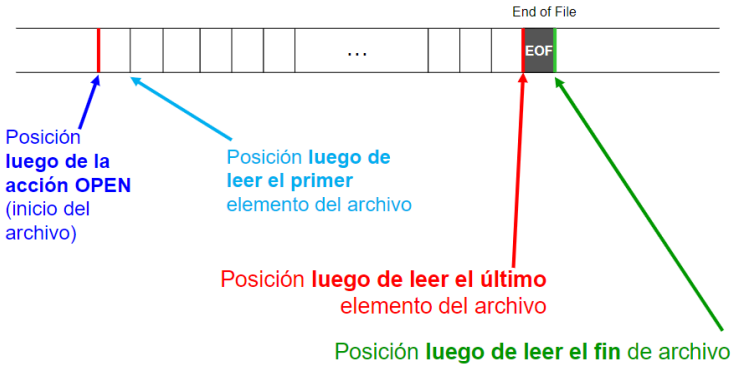
Librería <fstream>

- Nos provee de objetos que serán de entrada [*ifstream*], de salida [*ofstream*] o ambas [*fstream*].
- Todos los archivos disponen de operaciones para abrirlos, cerrarlos o saber si estamos en el final.

Información relativa a un archivo.

Los objetos del tipo *archivo* son *objetos lógicos* (variables de nuestro programa) que representan un archivo físico.

Para poder identificar de forma unívoca el *archivo físico* que manejan, se necesita un nombre de archivo.



Tipos de archivos. { Se distinguen de acuerdo a la *codificación o formato* en que almacenan la información }

- Archivos de texto
- Archivos binarios

Ambos almacenan datos usando un código binario, *la diferencia está en lo que representa cada uno*.

Archivos de texto.

- La información se almacena como una secuencia de caracteres.
- Cada carácter individual se almacena utilizando una codificación estándar (usualmente ASCII). Como es un formato estandarizado permite que otros programas diferentes de aquel que creó el archivo puedan entender y procesar su contenido.
- Los programas en código fuente codificados en un lenguaje de programación suelen ser almacenados como archivos de texto. Sin embargo, el resultado de compilar estos programas fuente a programas ejecutables se almacena en archivos binarios.

- Los archivos de texto al fin y al cabo son también archivos binarios, pero los tratamos como un tipo de archivo distinto porque los podemos ver, interpretar y editar desde un editor de texto.

Archivos binarios.

- La información se almacena con el mismo formato codificación utilizada por el compilador C++ para almacenar sus datos primarios en memoria RAM:
  - Los *números* aparecen en su forma binaria.
  - Los *caracteres* conservan su forma en ASCII.
- La ventaja de los archivos binarios es su *compactibilidad*: usan menos espacio para almacenar más números utilizando su *código binario* en lugar de valores de *caracteres individuales*.
- Están concebidos para ser procesados automáticamente por programas que conocen su formato interno. Es decir, otros programas externos no pueden interpretar la información de un archivo binario.
- El procesamiento de archivos binarios es más eficiente que el de archivos de texto porque la información está representada directamente en código binario (como se encuentra en la memoria principal). De esta forma se evita la pérdida de tiempo que ocasionaría su conversión a un formato estándar (ASCII, UTF-8, etc), como ocurre con los archivos de texto.
- **ej.** Si queremos guardar el número 123456 en un archivo de texto, *cada dígito ocupará 8 bytes*. Mientras que si lo queremos almacenar en un archivo binario, el número 123456 ocupará 4 bytes (lo que ocupa un *int* en C++).

Clasificación según tipo de acceso:

1. En función del flujo de los datos:
   a. De entrada
   b. De salida
   c. De entrada | salida
2. Dependiendo de cómo se accede a los datos en sí:
   a. Secuencial: El orden de acceso a los datos está determinado. Primero se accede al primer elemento y luego se puede ir accediendo a los siguientes, de uno en uno.
   b. Directo | Aleatorio: Se puede acceder de forma directa a un elemento concreto del archivo.

Entrada y salida de información.

- La (entrada | salida) de datos es a través de los *flujos estándares* de entrada y salida (*cin* | *cout*).
- Todo lo visto respecto a la entrada y salida básica con los flujos estándares, o la entrada y salida de cadenas, también es aplicable a los flujos de entrada y salida vinculados a archivos.
- *open* para archivos de salida: *si el archivo no existe lo crea*, y *si ya existe lo sobreescribe vacío*.

Comprobar la apertura del archivo.

Un archivo físico que *se haya intentado abrir y que no exista no tendrá* su correspondiente *archivo lógico asociado*.

- Para comprobar se usa la instrucción *fail* que consulta si el archivo falló
  $if(archivoTexto.fail())\{ / \ldots / \}$ // *fail devuelve true si el archivo no se abrió*.
- También se puede hacer con un *if* sobre la variable del tipo archivo
  $if(!archivoTexto)\{ / \ldots / \}$ // *Se cumple* la condición si el archivo *no se ha abierto*.

Lectura { Con el operador ">>" } -> $archivoTexto >> variableAIngresar;$

Igual que con las instrucciones de entrada estándar pero cambiando el flujo *cin* por el *nombre de la variable del tipo ifstream*.
Dicha operación pone en la *variableAIngresar* el siguiente elemento leído del archivo.

- Por defecto, el operador ">>" *ignora los espacios en blanco, tabulaciones y saltos de línea*.
  - Para que no los ignore: $archivoTexto.unsetf(ios::skipws);$
  - Para que los vuelva a ignorar: $archivoTexto.setf(ios::skipws);$

Lectura { Con *get* } -> $fichero.get(car, numCar, delimitador);$

Si se quiere leer todo el archivo tal y como está, sin saltearse espacios, tabulaciones, etc. se puede usar *get*. Se puede llamar de 3 formas:

- $int\ get();$ // Lee un sólo carácter y lo retorna. No se salta espacios en blanco.
- $get(char\&\ car);$ // Lee un sólo carácter de un flujo de entrada y lo pone en la variable *car*.
- $get(char\ *\ cad,\ int\ numCar,\ char\ delimitador = '\backslash n');$ // Lee un número determinado de caracteres dado en *numCar* y termina si encuentra un salto de línea. Los guarda en la cadena *cad*.
  - Se leerán como máximo *numCar-1* caracteres.
  - El delimitador es opcional. Por defecto es el salto de línea. Si se encuentra el delimitador, no es extraído de la secuencia de entrada, sino que permanece ahí como el siguiente carácter a ser extraído.

Lectura { Con *getline* } -> $fichero.getline(car, numCar, delimitador);$

Permite la lectura de toda una línea del archivo.

$$getline(char\ *\ cad,\ int\ numCar,\ char\ delim = '\backslash n');$$

Almacena en la cadena *cad* los caracteres del archivo hasta que se cumpla una de las siguientes:

- Haya leído *numCar-1* caracteres
- Encuentre el final del archivo
- Encuentre el delimitador
  - A diferencia del *get*, si se encuentra el delimitador, se extrae de la secuencia de entrada pero es descartado, es decir, no se guarda en la *cad*.

Tratamiento de Archivos.

1. Incluir la librería <fstream>
2. Usar el espacio de nombre std.
3. Declarar las variables que actuarán como manejadores de archivos.

4. Abrir el flujo de datos vinculando la variable correspondiente con el archivo especificado. Toda transferencia de información entre el programa y el archivo se realizará a través de la variable que ha sido vinculada a dicho archivo.
5. Comprobar que la apertura del archivo se realizó correctamente.
6. Realizar la transferencia de información { entrada | salida }. Normalmente se realizan mediante un *proceso iterativo*. Para la entrada dicho proceso suele requerir la lectura de todo el contenido del archivo.
7. Comprobar que el procesamiento del archivo del paso 6 se realizó correctamente.
    a. Para el caso del *procesamiento de entrada* consiste en comprobar si el estado de la variable manejador indica que *se ha alcanzado el final del archivo*.
    b. Para el caso del *procesamiento de salida* consiste en comprobar si el estado de la variable manejador indica que *se ha producido un error de escritura*.
8. Finalmente, cerrar el flujo para liberar la variable manejador de su vinculación con el archivo.
    a. En caso de no cerrar el flujo, éste será cerrado automáticamente cuando termine el ámbito de vida de la variable manejador del archivo.

Tratamiento de ficheros.

1. Apertura del archivo: Se mapea el objeto lógico (variable de nuestro programa) con el objeto físico (archivo en disco). El modo de apertura depende de si el archivo es de entrada | salida.
2. Acceso al archivo: Etapa donde se van leyendo | escribiendo los elementos.
3. Cierre del archivo: Actualiza el archivo y elimina la información relativa a él que se haya creado en el momento de abrirlo, tanto por parte del compilador como del sistema operativo.

- Para abrir un fichero se usa la instrucción *open* y como parámetro se pasa el nombre del fichero y el modo de apertura.
- Para cerrar un fichero se usa la instrucción *close* sin ningún argumento.

Archivos de Acceso Directo o Aleatorio.

Los archivos de *acceso secuencial* no son apropiados para las aplicaciones donde un registro en específico se debe localizar inmediatamente. Para eso usamos archivos de acceso aleatorio, cuyos registros individuales se pueden acceder de manera directa y rápida, sin tener que buscar en otros registros.

C++ *no impone una estructura sobre un archivo de acceso aleatorio*. La *aplicación* que desee usar este tipo de archivos *debe crearlas*. El método más sencillo es requerir que todos los registros en el archivo sean de la misma longitud fija. De este modo, es más fácil para el programa calcular la ubicación exacta de cualquier registro al inicio del archivo facilitando el acceso inmediato a registros específicos, incluso en archivos extensos.

Archivos binarios: Función de lectura -> $istream\&\ read\ (char\ *\ s,\ streamsize\ n);$

- Lee un *bloque de datos* desde un archivo binario.
- Extrae *n* caracteres (indicados en el segundo parámetro) desde un archivo de entrada y los almacena en la cadena de caracteres *s* (indicada en el primer parámetro).
- Para leer un tipo de dato que *no sea una cadena* se deberá hacer un *casting*.
- Esta función copia un bloque de datos sin importar el contenido (no importa si tiene caracteres nulos o de final).

Archivos binarios: Función de escritura -> $ostream\&\ write\ (const\ char\ *\ s,\ streamsize\ n);$

- Escribe un *bloque de datos* en un archivo binario.

- Inserta *n* caracteres (indicados en el segundo parámetro) en el archivo de salida, desde la cadena de caracteres *s* (indicada en el primer parámetro).
- No valida contenido.

Funciones de Posicionamiento

- En el *acceso aleatorio (directo)*, cualquier carácter en el archivo abierto puede leerse en forma directa sin tener que leer primero en forma secuencial todos los caracteres almacenados antes que el.
- Para proporcionar *acceso aleatorio a los archivos*, cada objeto *ifstream* crea en forma automática un *marcador de posición de archivo*, **seekg/seekp**.
    - ***seekg ("seek get"):*** Permite posicionarnos en una posición aleatoria en el archivo.
        - Se puede aplicar sobre archivos de *entrada*.
        - Indica la posición del próximo *get*.
    - ***seekp ("seek put"):*** Indica la posición para el próximo *put* en archivos de *salida*.
    - Ámbas funciones tienen el mismo formato:

$$archivo.seekg(pos_type\ *\ posi)$$

Dónde *posi* indica la posición donde se hará la próxima lectura. Esta forma es siempre relativa al inicio del archivo.

- O bien también pueden tener este formato:

$$archivo.seekg(offType\ *\ off,\ ios_base::direcc)$$

Dónde *off* indica la cantidad de posiciones a desplazarse, y *direcc* indica desde dónde se va a desplazar.

- Tabla de lugares donde se van a realizar los desplazamientos:

Valor	Descripción
ios::beg	**El desplazamiento es relativo al inicio del fichero**
ios::end	**El desplazamiento es relativo al final del fichero**
ios::cur	**El desplazamiento es relativo a la posición actual en el fichero**

- Cada carácter en un archivo de datos se localiza por su posición en el archivo. Se hace referencia a la posición de un carácter como su *desplazamiento desde el inicio del archivo*. El primer carácter tiene un desplazamiento de 0, el segundo un desplazamiento de 1, y así.

- El desplazamiento de la función *seek* es de *1 byte*.

- Este marcador es un *número entero* que representa un *desplazamiento* desde el principio de cada archivo e indica el *lugar desde donde se va a leer o escribir* el siguiente carácter.

Posición del Marcador de Posicionamiento: *tell*

La función *tell* sirve para saber en un momento determinado la *posición* en la que estamos.

- ***tellg():*** Devuelve la *posición del cursor* (o el valor de desplazamiento del marcador de posición) dentro del archivo de *lectura*.
- ***tellp():*** Lo mismo que *tellg();* pero para archivos de *escritura*.
    - ej. Si se leyeron 10 caracteres desde un archivo de entrada "archivo_entr", la siguiente sentencia devolverá el número 10:

$$archivo_entr.tellg();$$

Esto significa que el siguiente carácter que se va a leer está desplazado 10 posiciones desde el inicio del archivo, y es el undécimo carácter.